2025 IEEE International Conference on Robotics and Automation (ICRA)
May 19-23, 2025. Atlanta, USA

Enhancing Multi-Agent Systems via Reinforcement Learning with LLM-based Planner and Graph-based Policy

Ziqi Jia[1,2], Junjie Li[1,3], Xiaoyang Qu[1], Jianzong Wang[1,*]

***Abstract*— Multi-agent systems (MAS) have shown great potential in executing complex tasks, but coordination and safety remain significant challenges. Multi-Agent Reinforcement Learning (MARL) offers a promising framework for agent collaboration, but it faces difficulties in handling complex tasks and designing reward functions. The introduction of Large Language Models (LLMs) has brought stronger reasoning and cognitive abilities to MAS, but existing LLM-based systems struggle to respond quickly and accurately in dynamic environments. To address these challenges, we propose LLM-based Graph Collaboration MARL (LGC-MARL), a framework that efficiently combines LLMs and MARL. This framework decomposes complex tasks into executable subtasks and achieves efficient collaboration among multiple agents through graph-based coordination. Specifically, LGC-MARL consists of two main components: an LLM planner and a graph-based collaboration meta policy. The LLM planner transforms complex task instructions into a series of executable subtasks, evaluates the rationality of these subtasks using a critic model, and generates an action dependency graph. The graph-based collaboration meta policy facilitates communication and collaboration among agents based on the action dependency graph, and adapts to new task environments through meta-learning. Experimental results on the AI2-THOR simulation platform demonstrate the superior performance and scalability of LGC-MARL in completing various complex tasks.**

I. INTRODUCTION

Recently, there has been a growing interest in multi-agent systems (MAS) due to their immense capability to handle various tasks [1]–[4]. These systems are capable of handling simpler tasks like cleaning and surveillance, as well as more complex missions such as search and rescue missions or environmental surveillance [5]–[7]. However, coordinating multiple agents and ensuring safety—particularly in terms of collision avoidance [8]—remain major challenges. The complexity of these systems grows as the number of agents increases.

Multi-Agent Reinforcement Learning (MARL) provides a promising framework for agent collaboration. In this framework, agents learn optimal strategies by interacting with their environment, which enhances their ability to work together [9], [10]. For example, MAPPO [11] uses a centralized critic to facilitate coordination among agents, while QMIX [12] learns a global joint action value function by combining local Q-values to achieve multi-agent coordination. Despite the progress made, MARL still faces significant challenges when applied to real-world tasks. MARL struggles to execute complex human instructions and requires manual setting of task objectives as well as the design of intricate reward functions. These settings need to be adjusted as the scenarios change, which puts a heavy demand on the user's depth of understanding and experience with MARL.

The development of Large Language Models (LLMs) has introduced unprecedented reasoning and cognitive capabilities to robotics [13]–[16]. LLMs enhance the understanding of human intent and facilitate the prediction and execution of complex task sequences, improving the naturalness and effectiveness of human-machine interaction [17]–[21]. In multi-agent systems, the application of LLMs has shown great potential by fostering communication and cooperation among agents, leading to more efficient collaborative workflows [22]. However, current LLM-based MAS often rely heavily on dialogues between LLMs [23], which presents challenges for resource-constrained small robots. Additionally, in dynamic environments that require quick responses, this reliance on extensive dialogue can hinder the system's scalability and efficiency.

To address the aforementioned challenges and effectively integrate LLMs with MARL for multi-agent tasks, we propose the LLM-based Graph Collaboration MARL (LGC-MARL) framework. This framework leverages LLMs to decompose complex tasks into a series of low-level subtasks, and employs an action dependency graph to guide communication and cooperation among MARL agents. The underlying idea is that LLMs, equipped with rich world knowledge, can comprehend tasks and make plans based on the environment. By implicitly guiding the decision-making dependencies of MARL agents through the low-cost graph communication, LLMs enable MARL to iteratively learn optimal multi-agent collaboration policies through interaction with the environment. We introduce a critic-equipped LLM planner to extract environmental and task information, generate a sequence of subtasks, and employ another critic LLM module to verify the correctness of these subtasks, mitigating the hallucination problem of LLMs. To address the over-reliance on dialogues when using existing LLM data, we constrain the LLM's output to a graph format and adopt a graph-based collaboration meta policy for MARL agents to coordinate their actions, enabling flexible and efficient handling of LLM instructions. To tackle the challenging problem of designing reward functions in MARL, we propose an LLM-based reward function generator and carefully design prompts to guide the LLM to consider the problem from a MAS perspective, thus generating reward functions

[1] Ping An Technology (Shenzhen) Co., Ltd., Shenzhen, China
[2] Tsinghua Shenzhen International Graduate School, Tsinghua University, Shenzhen, China
[3] Huazhong University of Science and Technology, Wuhan, China
[*] Jianzong Wang is the corresponding author, jzwang@188.com

2025 IEEE International Conference on Robotics and Automation (ICRA) | 979-8-3315-4139-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICRA55743.2025.11127486

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:46:35 UTC from IEEE Xplore. Restrictions apply.

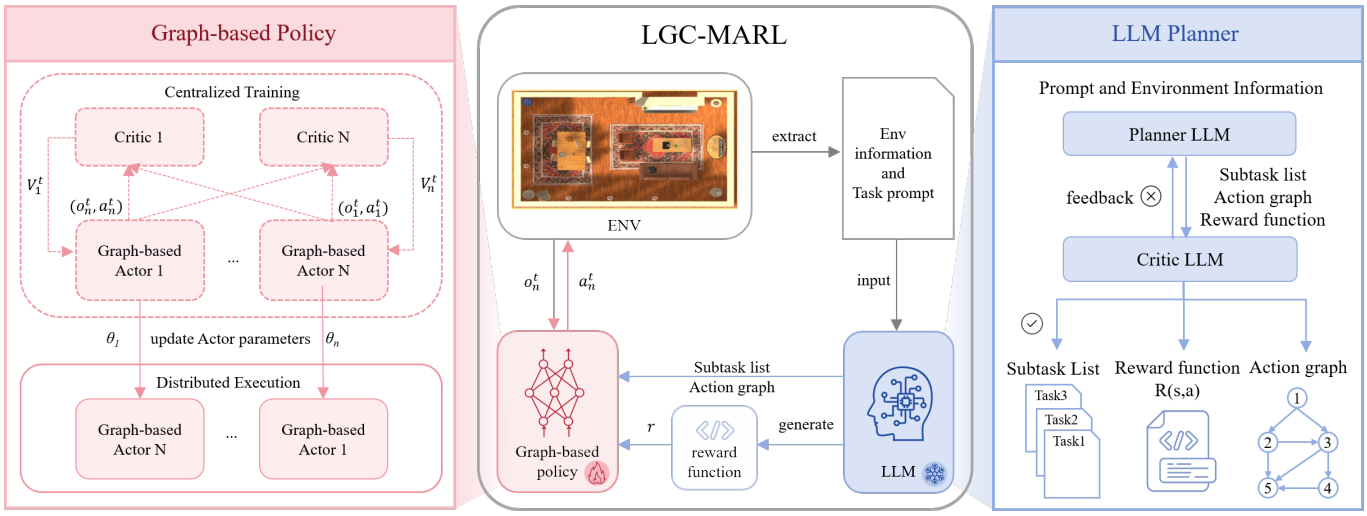


Fig. 1. The illustration of the proposed framework. The sub-figure in the middle shows the LGC-MARL framework, with the two key modules showcasing the environment interactions and the prompt input, the bottom two parts highlighting the graph-based policy (in red) and the LLM planner (in blue). When a new task is encountered, the two surrounding boxes reveal their internal workflows.

that promote agents collaboration. Our main contributions include:

- We have developed the LGC-MARL framework, integrating LLMs with MARL. By leveraging LLMs to decompose complex tasks into subtasks and employing action dependency graphs to guide inter-agent communication and cooperation, our method enables MARL agents to iteratively learn optimal collaborative strategies. Furthermore, by benefiting from the rich world knowledge and planning capabilities of LLMs, our approach enhances the efficiency of multi-agent tasks.
- We introduce a graph-based collaboration meta policy and an LLM-based reward function generator for MARL. Therefore, our agents can collaborate efficiently guided by an action dependency graph, and rapidly adapt to new task environments through meta-learning. Furthermore, our LLM-based reward generator tailors reward functions to the specific dynamics of MAS, enabling efficient learning and collaboration in MARL.
- Compared to existing methods, LGC-MARL exhibits significant advantages in terms of task complexity, adaptability, and collaborative efficiency.

II. RELATED WORKS

A. LLM for Multi-agent Systems

DyLAN [24] introduces a dynamic adjustment mechanism, allowing LLM agents to adapt their interaction modes based on real-time performance and task demands. EMMA [25] advances this concept by using centralized evaluation and natural language communication to fine-tune LLMs online, simplifying coordination among multiple agents. Chen et al. [26] studied how LLMs help multi-agent systems reach consensus, examining the impact of agent attributes, numbers, and network topology on the consensus process. CoELA [27] offers a modular approach, integrating LLMs into perception, memory, communication, and planning modules, facilitating collaborative task completion among agents. SMART-LLM [28] breaks down complex tasks into sequential stages, generating actions needed for complex goals. RoCo [23] equips each robotic arm with LLMs, enabling them to infer task strategies and generate trajectory points through dialogue. Most existing studies rely on LLM communication, limiting efficiency in dynamic environments. They also lack exploration, hindering optimal solution discovery. In contrast, our approach combines LLMs with MARL using graph structures, reducing overhead and improving decision quality for more resilient, exploratory multi-agent systems.

B. LLM for Reinforcement Learning

LLaRP [29] utilizes LLM to translate natural language instructions into task-specific unique languages and adopts an inside-out approach, avoiding direct exposure of RL strategies to natural language, thus promoting efficient learning. STARLING [30] leverages the LLM to translate natural language instructions into environment information and metadata, generating large amounts of text for RL agents to acquire necessary skills. In contrast, GLAM [31] employs the LLM as a policy to select the next action, interacting directly with the environment and training through online reinforcement learning. Plan4MC [32] acts as a high-level planner, utilizing the LLM to generate high-level action plans based on historical records and current environmental states, while combining independent low-level RL policies to execute specific actions. Eureka [33] proposes a self-reflective reward optimization algorithm that samples candidate reward functions from an encoded LLM and evaluates these functions through RL simulation, gradually improving the reward output. Meanwhile, Text2Reward [34] formulates adaptable dense reward functions as executable programs derived from environment descriptions. It refines rewards by implementing learning strategies within the environment and

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:46:35 UTC from IEEE Xplore. Restrictions apply.

incorporating human feedback.

III. METHOD

Our proposed framework is shown in Fig. 1, with the following process: First, the LLM generates a sequence of plans and a dependency graph based on environmental information. These outputs are then validated and refined by a critic model to ensure their feasibility and effectiveness. Once approved, the plan sequences and dependency graph are passed to a MARL policy. The MARL policy interacts with the environment and receives rewards from a reward function generated by the LLM. This allows the agents to optimize their decisions and improve performance over time.

A. Problem Setup

We model the scenario as a Multi-Agent Markov Decision Process (MMDP) denoted by $(S, \{A^i\}_{i=1}^N, \mathbb{P}, R, \gamma)$. Here, S represents the set of global states. For each agent i, A^i stands for its action space. We define the joint action of all N agents as $\boldsymbol{a} = (a^1, \ldots, a^N)$. The state transition probability function is given by $\mathbb{P}$. Meanwhile, R represents the reward function, and γ is the discount factor.

Our objective is finding a joint policy π^* maximizing the expected cumulative discounted reward:

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \right], \tag{1}$$

where $\mathbb{E}$ represents taking the expectation and t denotes the time step.

B. LLM Planner and Critic Model

We adopt a pre-trained LLM as a planner, which generates a series of planning steps based on environmental observations and task descriptions. The generated planning sequence can be represented as a list $[\zeta_l]$, where $l = 1, \ldots, L$, representing specific actions or decisions at each step. Additionally, the LLM constructs a graph structure that describes the decision dependencies among agents based on the generated planning sequence and the current positions of the agents. This graph not only reflects the collaborative relationships between different agents but also aids in further optimizing the effectiveness and coordination of decisions. To ensure the executability of the generated planning sequence in practical operations, we introduce another LLM as a critic model to evaluate the plans produced by the planner. As shown in Fig. 2, when the critic model identifies factual errors in the planning sequence, it conveys error information back to the planner through a feedback mechanism. Upon receiving this feedback, the planner revises the initial plan, generating an improved planning sequence and an updated decision dependency graph. Through the interaction between the planner and the critic model, we achieve continuous optimization of the plans, ensuring their effectiveness and accuracy during execution.

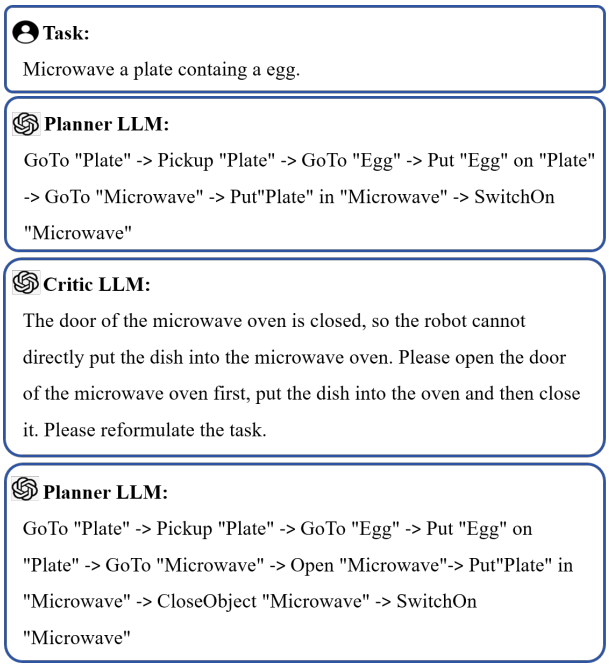


Fig. 2. Planner LLM and Critic LLM

C. LLM-based Reward Function Generator

The setting of the reward function is a traditional challenge in reinforcement learning. Compared to single-agent reinforcement learning, multi-agent reinforcement learning requires consideration of the interests of all agents when designing the reward function. Additionally, due to the non-stationary nature of the environment from each agent's perspective, crafting effective MARL reward functions is a formidable task that involves extensive debugging. As shown in Fig. 3, we input the abstracted environmental descriptions and task requirements into the LLM. The LLM is instructed to generate a reward function that considers both individual and team rewards for the agents, with a modular design. Although the LLM can help create complex reward functions, maintaining interpretability is crucial. Therefore, instead of directly providing the final reward function, we require the LLM to demonstrate its reasoning process. This is done by generating a series of intermediate steps, with each step explaining the rationale behind choosing a particular form of reward.

D. Graph-based Collaboration Meta Policy with LLM-based Graph Generator

LLMs possess strong capabilities for text understanding and generation, effectively capturing environmental states. We use an LLM to translate the state of the environment and the behavior of agents into textual form, then employ the LLM to generate text that describes decision dependencies. This leads to the construction of a directed acyclic graph (DAG) $\boldsymbol{G}$, representing the dynamic decision-making structure among agents. Finally, we convert DAG $\boldsymbol{G}$ into an

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:46:35 UTC from IEEE Xplore. Restrictions apply.

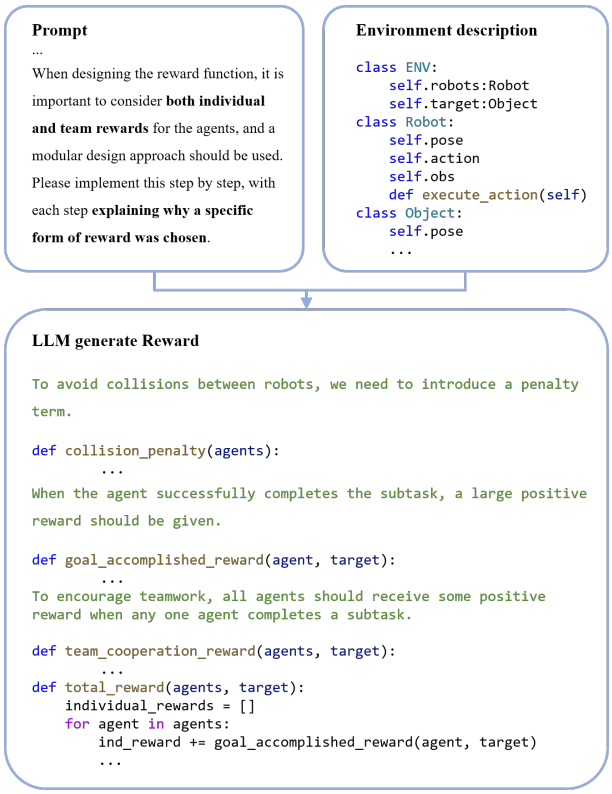


Fig. 3. LLM-based reward function generator

adjacency matrix $\boldsymbol{M}$. The formula is as follows:

$$\boldsymbol{M} = f(LLM(\{o^i\}_{i=1}^N)), \tag{2}$$

where $\{o^i\}_{i=1}^N$ denotes the environment observed by all agents, and f represents the function that converts the graph into an adjacency matrix.

To enable agents to quickly adapt to new environments and task scenarios, we incorporate meta-learning mechanisms into the graph-based reinforcement learning strategy. We define a set of tasks T, and the graph-based collaboration meta policy can be expressed as:

$$\pi_m = \prod_{i=1}^{N} \pi_m^i(a^i \mid o^i, a^{pt(i)\sim M}; T), \tag{3}$$

where π_m^i is the meta-learning policy of the i-th agent, which takes into account the information from the task set T, o^i represents the observation of the i-th agent, $pt(i)$ denotes the parent agents of agent i, and $a^{pt(i)\sim M}$ are the actions taken by these parent agents, with their order determined by the LLM. Our overall objective is to maximize the cumulative revenue, which can be expressed as:

$$\eta_m = \mathbb{E}_{\boldsymbol{a}\sim\pi(\cdot|s,M)}\left[\sum_{k=0}^{\infty}\gamma^k r\left(s_{t+k}, \boldsymbol{a}_{t+k}\right)\right]. \tag{4}$$

In a multi-agent environment, each agent i (where $i = 1, 2, \ldots, N$) adheres to a specific policy π_m^i. Collectively, these policies form the coordination strategy set of the entire system $\Pi = \{\pi_m^1, \pi_m^2, \ldots, \pi_m^N\}$. These policies are parameterized by a set of parameters $\theta = \{\theta^1, \theta^2, \ldots, \theta^N\}$. The gradient of expected return for each agent i can be written as follows:

$$\begin{aligned}\nabla_{\theta^i}\eta_m(\theta^i) &= \nabla_{\theta^i}\eta_m^i \\ &= \mathbb{E}_{a^i\sim\pi_m^i}\left[\nabla_{\theta^i}\log\pi_m^i(a^i \mid o^i, a^{pt(i)}; T) Q_{\pi_m}^i(s,a)\right],\end{aligned} \tag{5}$$

where $Q_{\pi_m}^i$ is the centralized action-value function, which can be updated to achieve iterative improvement as follows:

$$L(\phi) = \mathbb{E}_{(s,\boldsymbol{o},M,\boldsymbol{a})\sim\mathcal{D}}\left[\left(Q_{\pi_m}^i(s,\boldsymbol{a}) - y_m^i\right)^2\right], \tag{6}$$

$$y_m^i = r + \gamma \max_{u'} Q_{\pi_m^-}^i(s', a'), \tag{7}$$

where y_m^i is the learning target with meta-learning features, $Q_{\pi_m^-}^i$ is the target network parameterized by θ_-^i, and $\mathcal{D}$ is the experience replay buffer which records the experiences. The pseudocode is shown in **Algorithm 1**.

Algorithm 1: LGC-MARL.

Initialize LLM and graph-based collaboration meta policy π_m^i;
Initialize L, C, B and task set T;
Initialize the policy network parameters θ^i, ϕ^i, where $i = 1, ..., N$;
Initialize the target networks $\theta^{i'} = \theta^i$ and $\phi^{i'} = \phi^i$;
Initialize data buffer $\mathcal{D} = \{\}$;
for *each episode* **do**
  Get subtask list $[\zeta_l]$ from LLM;
  Get reward function R from LLM;
  Initialize $h_{0,\pi}^{(1)}, \ldots, h_{0,\pi}^{(N)}$ actor RNN states;
  Initialize $h_{0,V}^{(1)}, \ldots, h_{0,V}^{(N)}$ critic RNN states;
  for *each timestep* t **do**
    $\boldsymbol{G}_t = LLM(s_t)$;
    Get the adjacency matrix $\boldsymbol{M}_t = f_m(\boldsymbol{G}_t)$;
    for i *in order* $\boldsymbol{M}_t$ **do**
      $a_t^i, h_{t,\pi}^{(i)} = \pi_m^i(o_t^i, h_{t-1,\pi}^i, a_t^{pt(i)}; T)$;
      $v_t^i, h_{t,V}^{(i)} = Q_t^{\pi_m^i}(s_t^i, h_{t-1,V}^i; T)$;
      Receive reward r_t from R ;
      Observe next state o_{t+1};
      Add transition $\{o_t, a_t, r_t, o_{t+1}\}$ into $\mathcal{D}$;
    end
  end
  if $episodes > L$ **then**
    Sample a batch
    $\mathcal{B} = \{o_j, a_j, r_j, o_{j+1}\}_{j=0}^B \sim \mathcal{D}$;
    Update π_m^i using $\mathcal{B}$ and Eq. (5);
    Update ϕ^i using Eq. (6);
    Every C steps reset $\theta^{i'} = \theta^i$ and $\phi^{i'} = \phi^i$ for $i = 1, ..., N$;
  end
end

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:46:35 UTC from IEEE Xplore. Restrictions apply.

IV. EXPERIMENT

A. Experiment Setup

AI2-THOR [35] is a 3D simulation environment for home robots. Within this simulator, Robots can perform actions such as 'MoveLeft', 'MoveAhead', 'PickupObject', and 'PutObject'. We selected 4 scenes from AI2-THOR and designed specific tasks for each. All LLMs used were GPT-4.

B. Evaluation metrics

To comprehensively evaluate the performance of each method, we selected three key metrics: Success Rate (SR), Average Completion Time (AT), and Normalized Token Cost (TC).

- **Success Rate**: A task is considered failed if there are collisions, interferences between agents, or if the task times out. By calculating the percentage of successfully completed tasks, we can directly assess the robustness and stability of each method.
- **Average Completion Time**: This metric measures the average time(in seconds) taken by agents to complete a task. A shorter time indicates higher efficiency of the agent and indirectly reflects the optimization level of the algorithm.
- **Normalized Token Cost**: To better compare the language model resource consumption of different methods, we normalized the token consumption [5]. The specific calculation formula is as follows:

$$\hat{w}_i = \frac{w_i}{min(W)}, \tag{8}$$

where W is the set of token costs, i-th represents the token cost of the i-th method, $\hat{w}_i$ is the normalized metric value, and the best possible value for $\hat{w}_i$ is 1.0.

C. Results and Analysis

Comparison of Different LLM Methods. As shown in Table I, we conducted a comprehensive comparison between LGC-MARL and two prevalent LLM-based multi-agent methods: centralized LLM [28] and LLM dialogue [23]. Centralized LLM employs a single LLM to plan actions for all agents, limiting inter-agent collaboration and resulting in the lowest task success rate due to its inability to account for individual agent perspectives and dynamic environmental changes. In contrast, the LLM dialogue approach equips each agent with an independent LLM to facilitate coordination through dialogue, which enhances collaboration by enabling agents to exchange information and negotiate strategies. However, this approach also leads to the longest task completion time due to the computational overhead of natural language processing and the excessive token consumption from multi-round dialogues, which can become particularly problematic in complex scenarios. Furthermore, the increased number of dialogue rounds can exacerbate the risk of LLM hallucinations, as each interaction introduces opportunities for misinterpretation or generation of inconsistent information, further reducing task success. In comparison, our proposed LGC-MARL method demonstrated superior performance across all metrics. Whether it's task success rate, average completion time, or token consumption, our method consistently outperformed the baselines by leveraging structured communication and efficient decision-making mechanisms. This indicates that LGC-MARL effectively balances collaboration, efficiency, and resource utilization, making it a robust solution for multi-agent systems in diverse applications.

Ablation Study. To delve deeper into the contributions of each module within the LGC-MARL framework, we conducted ablation studies. By removing different modules, we constructed three variants: LGC-MARL (wo critic) without the LLM critic, LGC-MARL (wo reward) without the LLM-generated reward function, and LGC-MARL (wo graph) where the LLM directly interacts with MARL without a graph structure.

As shown in Table I, LGC-MARL (wo critic) experienced a significant drop in task success rate due to the LLM planner making more errors without the guidance of the LLM critic. LGC-MARL (wo graph) suffered from increased communication overhead and decreased inter-agent collaboration efficiency due to direct interactions, leading to longer task completion times. LGC-MARL (wo reward) exhibited suboptimal performance across all metrics due to the imperfect reward function, resulting in a less optimized MARL policy.

These experimental results unequivocally demonstrate the effectiveness of each module in LGC-MARL. The LLM critic plays a pivotal role in enhancing decision accuracy, the graph structure effectively reduces communication overhead and improves collaboration efficiency, and the LLM-generated reward function provides high-quality feedback for MARL learning.

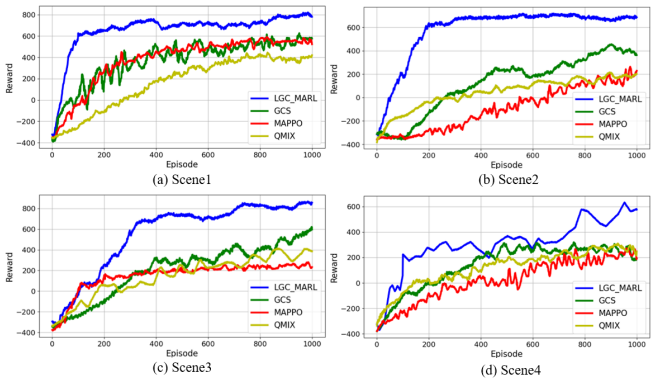


Fig. 4. Comparison of different MARL algorithms

Comparison of Different MARL Algorithms. To comprehensively evaluate the performance of LGC-MARL, we conducted comparative experiments against a suite of state-of-the-art MARL algorithms. Considering that MARL algorithms often struggle to effectively decompose high-level instructions when handling complex tasks, we pre-decomposed complex tasks into subtasks in our comparative experiments to facilitate fair comparisons between different algorithms.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:46:35 UTC from IEEE Xplore. Restrictions apply.

TABLE I

EVALUATION RESULTS ON FOUR SCENES. WE REPORTED THE AVERAGE SUCCESS RATE (↑) FOR EACH SCENE WITH A AGENT NUMBER OF 4 OVER MORE THAN 100 RUNS, THE AVERAGE TIME (↓) PER SCENE, AND THE NORMALIZED AVERAGE TOKEN COST (↓).

		Centralization LLM	LLMs Dialog Dialog	LGC-MARL (wo critic)	LGC-MARL (wo reward)	LGC-MARL (wo graph)	LGC-MARL
Scene1	Success Rate	0.6	0.68	0.76	0.89	0.73	**0.92**
	Average Time	99.59 ± 5.32	154.54 ± 9.74	68.61 ± 2.09	73.15 ± 3.39	78.95 ± 3.96	**66.47 ± 1.82**
	Token Cost	6.3	10.7	1.3	**1**	2.2	1.80
Scene2	Success Rate	0.54	0.61	0.7	0.84	0.67	**0.88**
	Average Time	107.74 ± 6.07	189.21 ± 11.82	80.94 ± 2.57	84.70 ± 3.46	88.24 ± 4.02	**78.76 ± 2.42**
	Token Cost	7.8	14.2	2.6	2.3	3	**2.1**
Scene3	Success Rate	0.57	0.64	0.74	0.86	0.69	**0.90**
	Average Time	105.11 ± 5.83	182.31 ± 10.57	77.72 ± 2.63	80.08 ± 3.58	86.01 ± 3.35	**75.93 ± 2.23**
	Token Cost	7.5	13.6	2.4	**2.1**	3.4	2.6
Scene4	Success Rate	0.51	0.60	0.71	0.8	0.65	**0.87**
	Average Time	115.85 ± 6.56	205.69 ± 11.31	**85.75 ± 2.81**	89.31 ± 4.24	95.37 ± 3.50	**84.81 ± 2.65**
	Token Cost	8.8	16.3	4.2	3.8	5.1	**3.5**

As illustrated in Fig. 4, our LGC-MARL algorithm consistently outperforms other baseline algorithms across various tasks, achieving significantly higher convergence rewards. In contrast, while classical MARL algorithms like QMIX and MAPPO can enable multi-agent collaboration, they fail to fully consider the interdependence of actions among agents, leading to suboptimal cooperation in complex scenarios and hindering the achievement of higher collaborative rewards. Although GCS [36] considers the collaborative relationships among agents to some extent, its relatively simple mechanism for generating action collaboration graphs limits its ability to fully leverage environmental information, resulting in slower convergence and less robust learned cooperative policies.

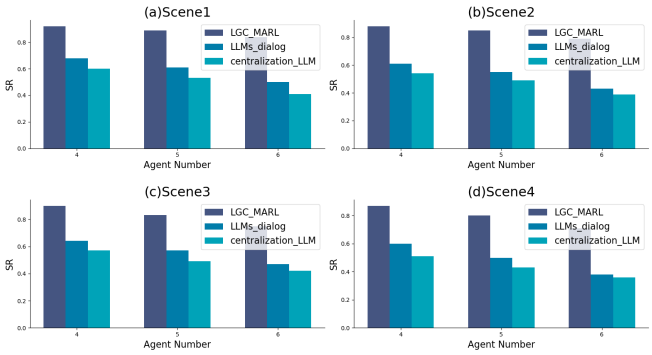


Fig. 5. Comparison of different agent numbers

Comparison of Different Agent Numbers. To delve deeper into the scalability of different approaches when handling multi-agent systems, we systematically investigated the impact of the number of agents on algorithm performance. As shown in Fig. 5, as the number of agents increases, both centralized LLM and LLM dialogue methods experience a significant decline in success rate. This is primarily because centralized LLM struggles to effectively manage the complex interactions among a large number of agents, while the LLM dialogue method is prone to lengthy dialogue cycles, leading to inefficient decision-making.

In contrast, our LGC-MARL method exhibits stronger robustness when the number of agents increases, with a smaller decrease in success rate. This indicates that LGC-MARL can effectively handle the complexity of multi-agent systems and maintain high performance even as the number of agents grows.

V. CONCLUSIONS

To enhance the collaboration efficiency and reduce communication overhead in multi-agent systems, we propose LGC-MARL, a framework that integrates MARL with LLMs. The framework consists of two components: an LLM planner and a graph-based collaboration meta policy. The LLM planner, equipped with rich world knowledge, can decompose complex tasks into a series of executable subtasks and generate an action dependency graph based on the environment. To mitigate the hallucination problem of LLMs and improve task success rates, we equip the LLM planner with an LLM critic to evaluate the rationality of the generated tasks and provide feedback. The graph-based collaboration meta policy guides agents to collaborate based on the action dependency graph and quickly adapts to new task environments through meta-learning. And LLM-based reward function generator generates reward functions tailored to the characteristics of multi-agent systems, ensuring efficient policy learning. Extensive experiments conducted on various scenarios in the AI2-THOR simulation platform demonstrate that LGC-MARL outperforms existing LLM-based and MARL-based methods. While maintaining low token consumption, LGC-MARL consistently achieves high success rates on tasks.

VI. ACKNOWLEDGEMENTS

This paper is supported by the Key Research and Development Program of Guangdong Province under grant No.2021B0101400003. Corresponding author is Jianzong Wang from Ping An Technology (Shenzhen) Co., Ltd.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:46:35 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] S. S. O. Venkata, R. Parasuraman, and R. Pidaparti, "Kt-bt: A framework for knowledge transfer through behavior trees in multirobot systems," *IEEE Transactions on Robotics*, 2023.

[2] W. Liu, K. Leahy, Z. Serlin, and C. Belta, "Robust multi-agent coordination from catl+ specifications," in *2023 American Control Conference (ACC)*. IEEE, 2023, pp. 3529–3534.

[3] J. G. Martin, F. J. Muros, J. M. Maestre, and E. F. Camacho, "Multi-robot task allocation clustering based on game theory," *Robotics and Autonomous Systems*, vol. 161, p. 104314, 2023.

[4] H. Chakraa, F. Guérin, E. Leclercq, and D. Lefebvre, "Optimization techniques for multi-robot task allocation problems: Review on the state-of-the-art," *Robotics and Autonomous Systems*, p. 104492, 2023.

[5] Y. Chen, J. Arkin, Y. Zhang, N. Roy, and C. Fan, "Scalable multi-robot collaboration with large language models: Centralized or decentralized systems?" in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 4311–4317.

[6] H. Zhang, J. Chen, J. Li, B. Williams, and S. Koenig, "Multi-agent path finding for precedence-constrained goal sequences," in *International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2022.

[7] J. Chen, Y. Gao, J. Hu, F. Deng, and T. L. Lam, "Meta-reinforcement learning based cooperative surface inspection of 3d uncertain structures using multi-robot systems," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 7201–7207.

[8] S. Yao, G. Chen, L. Pan, J. Ma, J. Ji, and X. Chen, "Multi-robot collision avoidance with map-based deep reinforcement learning," in *2020 IEEE 32nd International Conference on Tools with Artificial Intelligence (ICTAI)*. IEEE, 2020, pp. 532–539.

[9] K. Zhang, Z. Yang, and T. Başar, "Multi-agent reinforcement learning: A selective overview of theories and algorithms," *Handbook of reinforcement learning and control*, pp. 321–384, 2021.

[10] A. Oroojlooy and D. Hajinezhad, "A review of cooperative multi-agent deep reinforcement learning," *Applied Intelligence*, vol. 53, no. 11, pp. 13 677–13 722, 2023.

[11] H. Kang, X. Chang, J. Mišić, V. B. Mišić, J. Fan, and Y. Liu, "Cooperative uav resource allocation and task offloading in hierarchical aerial computing systems: A mappo-based approach," *IEEE Internet of Things Journal*, vol. 10, no. 12, pp. 10 497–10 509, 2023.

[12] T. Rashid, G. Farquhar, B. Peng, and S. Whiteson, "Weighted qmix: Expanding monotonic value function factorisation for deep multi-agent reinforcement learning," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 10 199–10 210, 2020.

[13] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, *et al.*, "Inner monologue: Embodied reasoning through planning with language models," *Conference on Robot Learning (CoRL)*, 2022.

[14] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 9493–9500.

[15] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, "Progprompt: Generating situated robot task plans using large language models," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 11 523–11 530.

[16] J. Wu, R. Antonova, A. Kan, M. Lepert, A. Zeng, S. Song, J. Bohg, S. Rusinkiewicz, and T. Funkhouser, "Tidybot: Personalized robot assistance with large language models," *Autonomous Robots*, vol. 47, no. 8, pp. 1087–1102, 2023.

[17] K. Lin, C. Agia, T. Migimatsu, M. Pavone, and J. Bohg, "Text2motion: From natural language instructions to feasible plans," *Autonomous Robots*, vol. 47, no. 8, pp. 1345–1365, 2023.

[18] Y. Ding, X. Zhang, C. Paxton, and S. Zhang, "Task and motion planning with large language models for object rearrangement," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 2086–2092.

[19] W. Yu, N. Gileadi, C. Fu, S. Kirmani, K. Lee, M. G. Arenas, H. L. Chiang, T. Erez, L. Hasenclever, J. Humplik, B. Ichter, T. Xiao, P. Xu, A. Zeng, T. Zhang, N. Heess, D. Sadigh, J. Tan, Y. Tassa, and F. Xia, "Language to rewards for robotic skill synthesis," in *Conference on Robot Learning (CoRL)*, 2023.

[20] Y. Chen, J. Arkin, C. Dawson, Y. Zhang, N. Roy, and C. Fan, "Autotamp: Autoregressive task and motion planning with llms as translators and checkers," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 6695–6702.

[21] A. Irpan, A. Herzog, A. T. Toshev, A. Zeng, A. Brohan, B. A. Ichter, B. David, C. Parada, C. Finn, C. Tan, *et al.*, "Do as i can, not as i say: Grounding language in robotic affordances," in *Conference on Robot Learning (CoRL)*, 2022.

[22] R. Chen, W. Song, W. Zu, Z. Dong, Z. Guo, F. Sun, Z. Tian, and J. Wang, "An llm-driven framework for multiple-vehicle dispatching and navigation in smart city landscapes," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 2147–2153.

[23] Z. Mandi, S. Jain, and S. Song, "Roco: Dialectic multi-robot collaboration with large language models," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 286–299.

[24] Z. Liu, Y. Zhang, P. Li, Y. Liu, and D. Yang, "Dynamic llm-agent network: An llm-agent collaboration framework with agent team optimization," *arXiv preprint arXiv:2310.02170*, 2023.

[25] Y. Yang, T. Zhou, K. Li, D. Tao, L. Li, L. Shen, X. He, J. Jiang, and Y. Shi, "Embodied multi-modal agent trained by an LLM from a parallel textworld," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2024, pp. 26 265–26 275.

[26] H. Chen, W. Ji, L. Xu, and S. Zhao, "Multi-agent consensus seeking via large language models," *arXiv preprint arXiv:2310.20151*, 2023.

[27] H. Zhang, W. Du, J. Shan, Q. Zhou, Y. Du, J. B. Tenenbaum, T. Shu, and C. Gan, "Building cooperative embodied agents modularly with large language models," *International Conference on Learning Representations (ICLR)*, 2024.

[28] S. S. Kannan, V. L. N. Venkatesh, and B. Min, "SMART-LLM: smart multi-agent robot task planning using large language models," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 12 140–12 147.

[29] A. Szot, M. Schwarzer, H. Agrawal, B. Mazoure, R. Metcalf, W. Talbott, N. Mackraz, R. D. Hjelm, and A. T. Toshev, "Large language models as generalizable policies for embodied tasks," in *International Conference on Learning Representations (ICLR)*, 2024.

[30] Y. Wu, S. Y. Min, S. Prabhumoye, Y. Bisk, R. R. Salakhutdinov, A. Azaria, T. M. Mitchell, and Y. Li, "Spring: Studying papers and reasoning to play games," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2024.

[31] T. Carta, C. Romac, T. Wolf, S. Lamprier, O. Sigaud, and P.-Y. Oudeyer, "Grounding large language models in interactive environments with online reinforcement learning," in *International Conference on Machine Learning (ICML)*. PMLR, 2023, pp. 3676–3713.

[32] P. BAAI, "Plan4mc: Skill reinforcement learning and planning for open-world minecraft tasks," *arXiv preprint arXiv:2303.16563*, 2023.

[33] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, "Eureka: Human-level reward design via coding large language models," *International Conference on Learning Representations (ICLR)*, 2024.

[34] T. Xie, S. Zhao, C. H. Wu, Y. Liu, Q. Luo, V. Zhong, Y. Yang, and T. Yu, "Text2reward: Automated dense reward function generation for reinforcement learning," *International Conference on Learning Representations (ICLR)*, 2024.

[35] E. Kolve, R. Mottaghi, W. Han, E. VanderBilt, L. Weihs, A. Herrasti, M. Deitke, K. Ehsani, D. Gordon, Y. Zhu, *et al.*, "Ai2-thor: An interactive 3d environment for visual ai," *arXiv preprint arXiv:1712.05474*, 2017.

[36] J. Ruan, Y. Du, X. Xiong, D. Xing, X. Li, L. Meng, H. Zhang, J. Wang, and B. Xu, "Gcs: Graph-based coordination strategy for multi-agent reinforcement learning," *International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2022.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:46:35 UTC from IEEE Xplore. Restrictions apply.